Minilistic MNIST Utilizing Modern Optimizations

CS 211 #18849

Brendan Tea, brendan.tea@bellevuecollege.edu,
Advisor: Professor Dr. Taesik Kim

Abstract: Value nodes with elementary operations can be built to create complex functions that achieve practical appliances. Although more memory intensive, this serves as a bare minimum for artificial intelligence architectures—that is without the computational optimizations. These value nodes hold a history of past operations and children nodes, creating a directed acyclic graph that allows us to calculate local gradients. Through these value nodes, we build up neurons, layers, and all the way up to a multilayer perceptron. With the MNIST dataset, we're able demonstrate the functionality of the value node multilayer perceptron using a training loop with gradient descent resulting in empirical results of 92∼% accuracy. Utilizing optimizations such as L2 normalization, adaptive moment estimation (Adam), cross entropy loss, and Gaussian error linear unit (GELU) resulted to lower loss and a final accuracy of 97∼%.

1 Introduction

Instead of using matrices and linear algebra to replicate a mulitlayer perceptron, we use value nodes. We define each fundamental operator—add, multiply, exponentialization, and trigonometric functions— alongside their partial derivatives with the chain rule. These individual value nodes will additionally hold previous children, creating a directed acyclic graph. All of this together recreates an autogradient engine vital for back propagation. In whole, we can create a multilayer perceptron, train it on the MNIST dataset, and obtain promising results.

2 Building Up

With these operators, we can define any function atomically. To build up to a multilayer perceptron. A neural network is composed of layers which are composed of neurons. This structure (with a non-linear function) allows the approximation of any shape it or desired behavior.

DEFINITION 2.1 (Neuron)**.** A neuron or perceptron is denoted by the following:

$$\mathrm{N} = \sum_{i=0}^{n} w_i x_i + b, \quad n \text{ being the input size.}$$

$$\mathrm{N} = \begin{bmatrix} x_0 & x_1 & \cdots & x_i \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_i \end{bmatrix} + \begin{bmatrix} b \end{bmatrix},$$

i being the number of inputs.

DEFINITION 2.2 (Layer)**.** A layer of neurons or a single layer perceptron is denoted by the following:

$$\mathrm{L} = \begin{bmatrix} N_0 \\ N_1 \\ \vdots \\ N_i \end{bmatrix}, \quad i \text{ being the amount of neurons.} \tag{2.1}$$

DEFINITION 2.3 (MLP)**.** A multi layer perceptron has multiple layers and it is denoted by the following:

$$\begin{aligned} \mathrm{MLP} &= \begin{bmatrix} L_0 & L_1 & \cdots & L_i \end{bmatrix} \\ &= \begin{bmatrix} \begin{bmatrix} N_0 \\ N_1 \\ \vdots \\ N_a \end{bmatrix} & \begin{bmatrix} N_0 \\ N_1 \\ \vdots \\ N_b \end{bmatrix} & \cdots & \begin{bmatrix} N_0 \\ N_1 \\ \vdots \\ N_c \end{bmatrix} \end{bmatrix}, \end{aligned}$$

a, b, c being the size of each layer.

Now that we have some infrastructure to work with, we have to show how forward pass would work. Imagine we have a 2 layer (1 input/middle, 1 output). For the input layer we shall take in two inputs and output 4. The output layer must take 4 and we will let it output 1.

This is how the math would work out.

$$X_0 = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}, L_0 = \begin{bmatrix} N_0 \\ N_1 \\ N_2 \\ N_3 \end{bmatrix}, N_i = \begin{bmatrix} x_0 & x_1 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} + \begin{bmatrix} b \end{bmatrix}$$

$$L_1 = \begin{bmatrix} N_0 \end{bmatrix}, N_0 = \begin{bmatrix} N_0 \\ N_1 \\ N_2 \\ N_3 \end{bmatrix}^\top \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \end{bmatrix}$$

$$= \begin{bmatrix} N_0 & N_1 & N_2 & N_3 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \end{bmatrix} + \begin{bmatrix} b \end{bmatrix}$$

basically linear regression but on steroids. We will forward pass inputs and receive an output. With the output, we will calculate the loss and run back propagation for the individual gradients and apply gradient descent on the parameters to reduce the loss value. In turn, this will result in a model that is better suited toward the result. Repeating this process will eventually lead to any function reaching a minimum.

2.1 Back Propagation

To actually train and get the wanted behavior from a function such as a MLP, we must obtain the direction of largest or smallest growth otherwise known as the gradient of the parameters. With the gradient, we can use that to move in a direction of lower loss. With a little bit of multi-variable calculus and the chain rule, we can obtain all of the local derivatives and in turn, their gradients. Here is the math behind it.

The most rudimentary case is to imagine taking derivatives at the root.

$$\frac{\partial L}{\partial c} = \frac{\partial L}{\cancel{\partial L}}^{1} \frac{\partial L}{\partial c} = \frac{\partial L}{\partial c} \tag{2.2}$$

However, imagine if we have a function of two variables with one being a composite. In this example, it should become clear that we will always calculate the outer derivative and multiply it by the inner derivative. At the start, we will simply take it's derivative and multiply it by one.

DEFINITION 2.4 (Chain Rule)**.** With that, back propagation is defined by the chain rule.

$$L(c, d) = c + d, \quad c(a) = a + b; \tag{2.3}$$

$$\implies \boxed{\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c}\frac{\partial c}{\partial a}} \tag{2.4}$$

Acculumating gradients:

$$L(c, d) = c(a) + d(a), \quad c(a) = a + b,\ d(a) = a + c \tag{2.5}$$

$$\implies \boxed{\frac{\partial L}{\partial a} = \frac{\partial L}{\partial c}\frac{\partial c}{\partial a} + \frac{\partial L}{\partial d}\frac{\partial d}{\partial a}} \tag{2.6}$$

DEFINITION 2.5 (Gradient)**.** We can take this farther and define the gradient of the loss function.

$$\boxed{\nabla L = \left\langle \frac{\partial L}{\partial a_0}, \dots \frac{\partial L}{\partial a_i} \right\rangle} \tag{2.7}$$

This is a very powerful tool and the next thing to tackle is how we order our function so that we can create an algorithm to start from the root and propagate back.

2.2 Topological Sort

With back propagation, we will need to be able to know what order to propagate. This is basically a job scheduling problem so topological sort will be the perfect algorithm for this. Topological sort works by applying a post-order depth first search on a directed acyclic graph and reversing the list. This can easily be done using a recursive formula to completely search each branch. Reversing should be extremely simple with a library.

Algorithm 2.1 DFS for Topological Sort

```
procedure DFS(v, topo, visited)
    if v ∉ visited then
        visited ← visited ∪ {v}
        for all child ∈ v.prev_children do
            DFS(child, topo, visited)
        end for
        append(topo, v)
    end if
end procedure
```

3 MNIST

First, we initialize an MLP with an input size and layer params. The layer params are simply the size of each layer.

After that, we will run a training loop. A training loop is simply composed of these fundamental steps:

1. Zero out all of the local gradients.
2. Feedforward the input through the MLP an obtain an output.

3. Calculate the loss with the output.

4. Potentially reduce the loss with gradient descent.

3.1 MNIST training loop

In the case with the MNIST dataset, we go through each component of the training loop one by one and build it. First we obtain the data as a 784 (28 x 28 image) vector and normalize every value which is to divide by 255. We obtain this number because the data is in the form of brightness values from 0-255 hence the normalization factor. We additionally obtain the proper correct label to check for the loss function. Now, feed forward and obtain an output vector. I decided for an 10 size output vector for each number from 0-9. We run this softmax to gain a "percentage" distrubution of numbers and to clamp them from 0-1.

I implemented both mean squared and cross entropy loss. In comparison with the one-hot label, we can compare that with our 10 size output vector. Mean squared simply sums all of the the squared difference between each index and normalizing it with the size. (Hence the name mean square.) Cross entropy takes the sum of the expected value multiplied by the log of the predicted output value. In the following section, we can compare both loss functions and their effectiveness.

Given here are the algorithms.

DEFINITION 3.1 (Mean Square Loss)**.**

$$\mathcal{L}(x) = \frac{\sum_i^n (\text{expected}_i - \text{predicted}_i)^2}{n} \tag{3.1}$$

DEFINITION 3.2 (Cross Entropy Loss)**.**

$$\mathcal{L}(x) = \frac{\sum_i^n (\text{expected}_i \cdot \log(\text{softmax}(\text{predicted}_i)))^2}{n} \tag{3.2}$$

3.2 Initial Results

Running the training loop resulted in slow convergence in loss and a peak accuracy of around 92∼%. Looking at the softmax output vector, it seems to have low confidence in its choices.

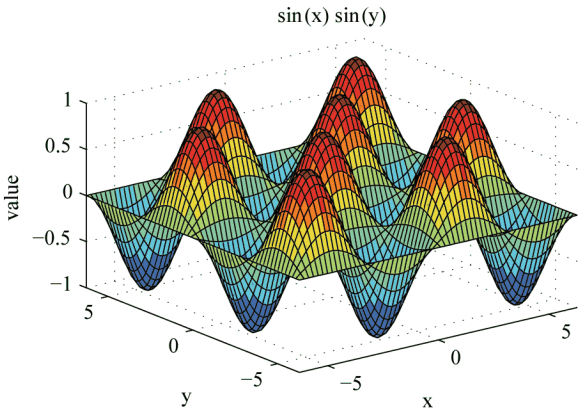

Figure 3.1: Accuracy and Loss versus batch

3.3 Optimizations

Optimizations were imperative to improving training speed and accuracy. For example, switching to cross entropy loss was better suited for the values than 1 as mean squared loss would make decimal values actually smaller. Additionally, using GELU instead of RELU helped reduce vanishing gradients. Something to realize is that the images are always centered and therefore the outer edge nodes should have no activation. Pruning out those values helped improve computer time. The most vital improvement was the Adam optimizer with two moments that accelerated convergence. All of this all together trained on 1 epoch resulted a peak accuracy of 92∼%.

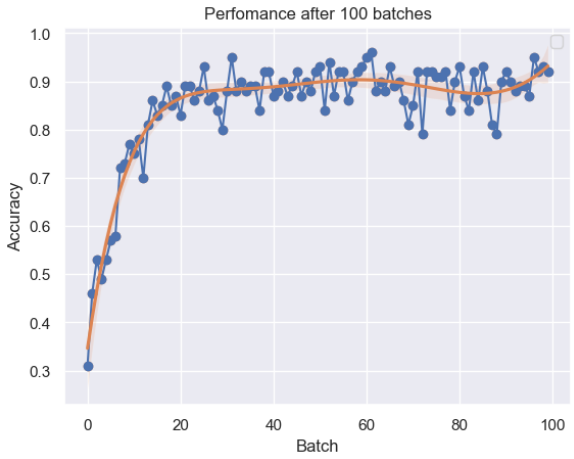

Figure 3.2: Accuracy and Loss versus batch

4 Conclusions

Ultimately, value nodes have shown their versatility in performing complex tasks. There are however some massive drawbacks as it is much more

computationally heavy to do individual calculations than with matrix operations. In the future with a matrix implementation, it may be better to utilize the Muon optimizer (Jordan et al. (2024)) alongside data standardization with synthetic data. With better generalizations with the dataset, we can push this minilistic, bare bones implementation to the limit. In whole, regardless of potential tweaks, this model's ability is not limited by its simplicity.

References

Jordan, K., Jin, Y., Boza, V., You, J., Cesista, F., Newhouse, L., & Bernstein, J. (2024). *Muon: An optimizer for hidden layers in neural networks.* Retrieved from `https://kellerjordan.github.io/posts/muon/`